Next.js Performance Optimization Notes

These detailed notes cover seven critical bottlenecks and their solutions to dramatically increase Next.js application speed, based on the provided video analysis. These guidelines can be used to inform your cloud code prompts and architectural decisions.

1. Maximize React Server Components

- **The Bottleneck:** Applying the "use client" directive to main pages forces the application to ship a larger JavaScript bundle to the browser and execute hydration, slowing down load times.
- **The Solution:** Leave main pages as standard React Server Components. Server components skip shipping JavaScript bundles and bypass the hydration step, making pages render significantly faster.
- **Implementation Rule:** Only use the "use client" directive on small, isolated components that explicitly require interactivity, such as those using the useState hook.

2. Implement the Next.js Image Component

- **The Bottleneck:** Using standard HTML <img> tags for large image files (e.g., 5MB) causes massive network payloads and slow page loads.
- **The Solution:** Replace <img> with the Next.js <Image> component from next/image. This component automatically optimizes image size, improves visual stability, and prevents layout shifts.
- **Implementation Rule:**
    - Always provide width and height properties to infer the correct aspect ratio and prevent layout shifting.
    - Add the image's hostname to the next.config.js file. This is required to prevent malicious actors from using your server resources to optimize their own images.

3. Replace Heavy Date Libraries with Native APIs

- **The Bottleneck:** Using libraries like moment.js adds significant bloat to the application bundle (e.g., 73KB minified and gzipped). This is especially detrimental to performance if deployed on serverless functions. AI coding tools often unnecessarily inject these libraries.
- **The Solution:** Delete imports for heavy date libraries and rely on native JavaScript APIs.

- **Implementation Rule:** Use the native Intl.DateTimeFormat API for date formatting, which handles standard requirements effectively while saving substantial bundle size.

4. Enable Streaming with React Suspense

- **The Bottleneck:** Slow backend data queries block the entire page from rendering—including static elements like navbars—until the data finishes loading.
- **The Solution:** Implement streaming to send parts of a dynamic route to the client as soon as they are ready.
- **Implementation Rule:** Wrap slow, data-fetching components inside React <Suspense> boundaries. Provide a fallback UI (like a loading skeleton). This allows static content to render instantly while dynamic data swaps in once it finishes loading.

5. Prevent Layout Data Fetching from Breaking Static Rendering

- **The Bottleneck:** Fetching user sessions or data on the server side within a layout file forces all child routes to opt into dynamic rendering. This disables Next.js's ability to pre-render pages as static content served from a global CDN.
- **The Solution:** For layouts wrapping pages that don't inherently need dynamic data, fetch session state on the client side instead.
- **Implementation Rule:** Mark UI components like Navbars with "use client" and utilize client-side hooks (e.g., useKindeBrowserClient) to fetch user sessions, allowing the underlying page routes to remain static. Handle the loading state by returning null briefly to avoid UI flashing.

6. Offload Route Protection to Middleware

- **The Bottleneck:** Calling a user authentication function (e.g., requireUser) inside a layout file to protect routes forces dynamic rendering for all enclosed pages.
- **The Solution:** Utilize Next.js Middleware to intercept incoming requests before they hit the layout or server logic.
- **Implementation Rule:** Create a middleware.ts file in the root directory to check for valid user sessions and handle redirects. Define a publicPaths array to exclude non-protected routes from the middleware checks. This preserves static rendering for unaffected routes.

7. Deduplicate Repeated Data Fetches with React Cache

- **The Bottleneck:** Fetching a user session or checking authorization individually within multiple data access layer functions can cause redundant requests (e.g., fetching the same user session 6 times during a single page render).
- **The Solution:** Use React's cache function to store the result of a data fetch or computation for the duration of a single render pass.
- **Implementation Rule:** Wrap the utility function (e.g., export const requireUser = cache(async () => {...})) in the cache function. This ensures that even if the function is called multiple times on one route, the network request is only executed once and the result is reused.